

# Solana integration: review and ideation

A plain-language review of the Solana code already in the project, the Solana-first plan, and how to bring them together.

## The short answer

**What exists:** a small, working bridge. When an order is accepted, delivered or settled, the API writes a record to a Solana program: fingerprints and a status, no money. It has been tested only against a fake network connection, was never run on devnet, and is not active in your live site.

**What the plan proposes:** a rebuild where Solana holds the money (escrow) and the shared trust state, buyers sign with wallets, and an indexer copies chain events into Postgres. So far it is documents only (solana-first/ has no code).

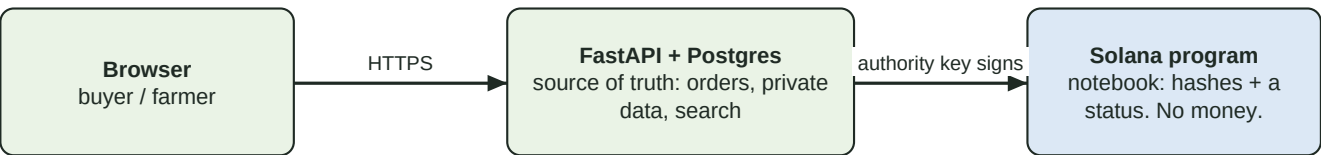
**Recommendation:** do not rebuild. Grow the existing program in three stages: make the audit trail real, then add escrow, then harden for mainnet. Each stage is useful on its own, so you can stop after any of them.

## Four words you need

|                      |                                                                                                                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Program</b>       | The code that runs on Solana (ours is written in Rust with the Anchor framework). Think of it as a tiny server nobody can secretly change.                                                                      |
| <b>Account / PDA</b> | Storage that belongs to a program. A PDA is an account whose address is calculated from a recipe (for us: the word 'settlement' plus a fingerprint of the order id), so every order has one predictable record. |
| <b>Hash</b>          | A 32-byte fingerprint of some data. We store the fingerprint on chain, not the photos or names, so privacy stays in our database but anyone can prove the data was not altered.                                 |
| <b>Escrow</b>        | Money held by a neutral vault until agreed conditions are met. Today there is no escrow: the chain stores facts, not funds.                                                                                     |

## The big picture: today versus the plan

**TODAY - the database is the boss, Solana is a notebook**



**THE PLAN - Solana holds the money and shared trust state, Postgres keeps everything else**

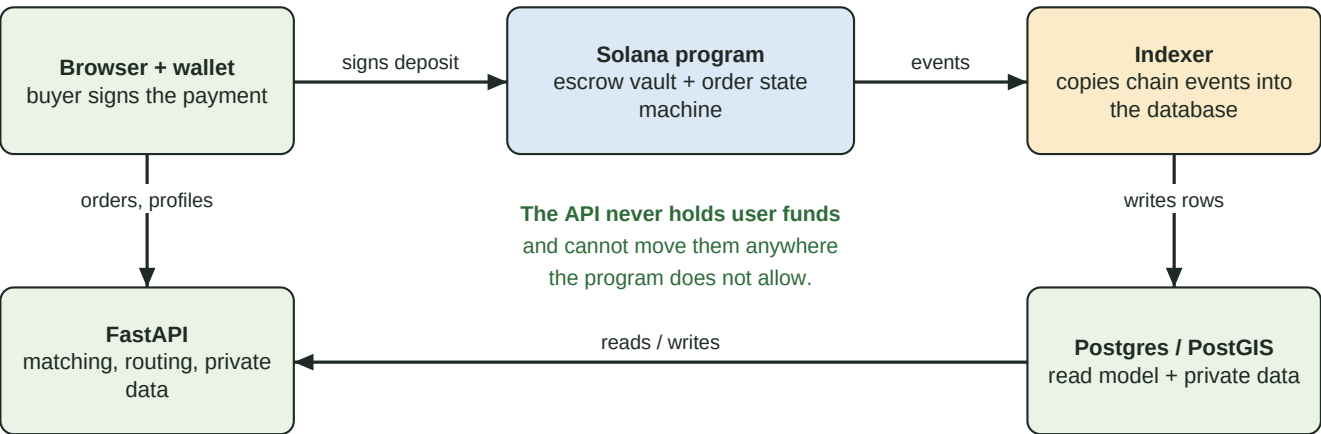


Figure 1. Boxes show who does what. Arrows show who talks to whom.

# How the current integration works

Files: `api/app/payments/` (queue and Python client) and `blockchain/solana/programs/marketplace-settlement/` (the program). The marketplace announces order events; the payments module listens and mirrors them to Solana.

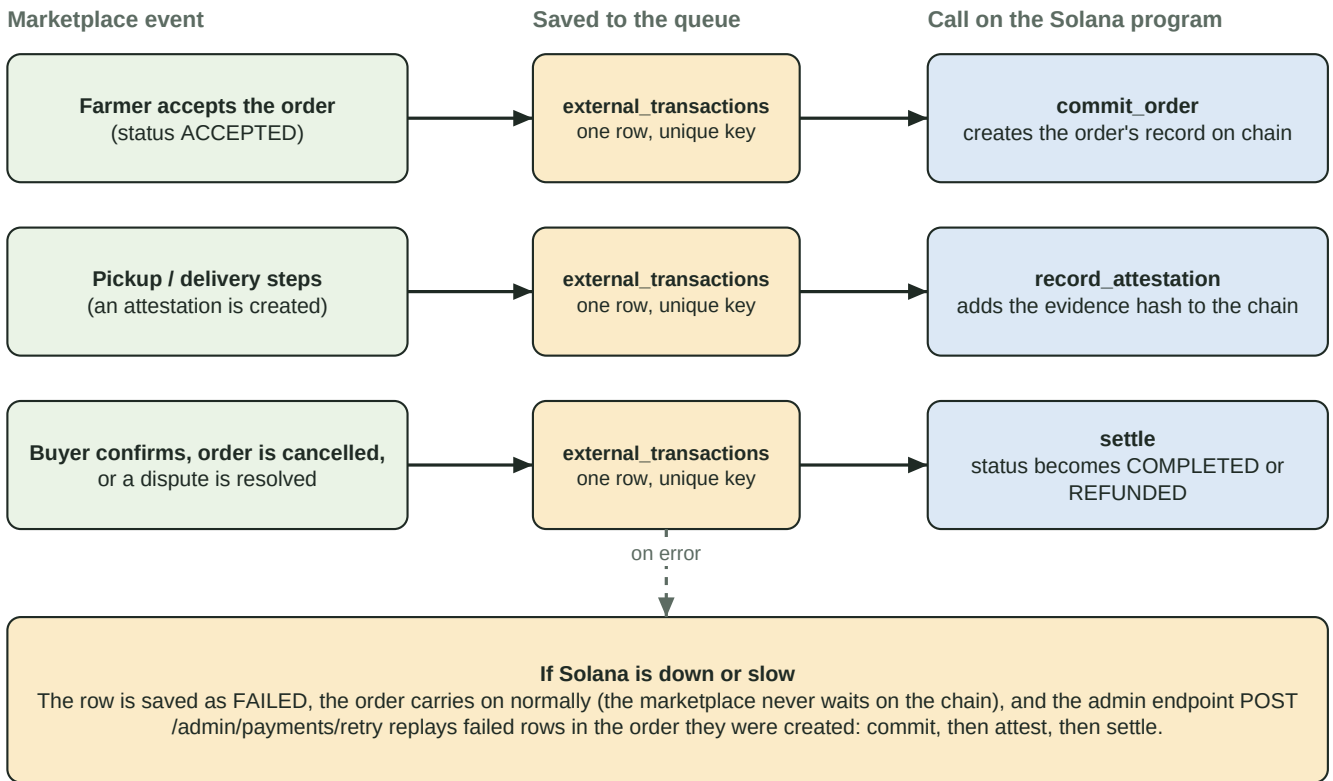
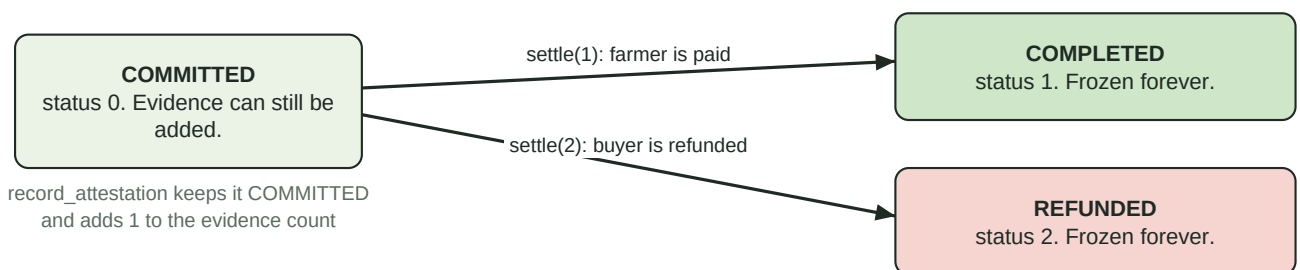


Figure 2. Three order events, three program calls. Every call goes through a saved queue row.

## What the program remembers about each order

Each order gets one record with: the platform authority, a fingerprint of the order id, the amount in cents, a status, how many pieces of evidence were added, and a rolling fingerprint of that evidence. Nothing else.

### Order record on chain: it can only move forward, once



### Evidence chain: each new value is sha256(previous value + new hash)

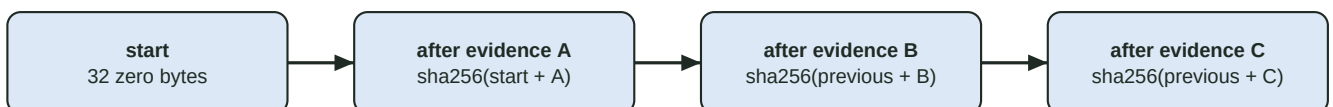


Figure 3. The status can move once, from COMMITTED to a final state. The evidence chain makes it impossible to edit or reorder earlier evidence without the later values changing.

# Review: what is good, and what to fix

## What is already solid

- Only hashes and a status go on chain, so no personal data is public or permanent.
- The program re-checks the order address, the signer and the status on every call, and refuses any change after settlement.
- A chain outage never blocks an order: failures are stored and retried in order.
- Each event has a unique key, so the same event is not queued twice.
- The payment interface is swappable (mock by default), so the marketplace does not depend on Solana.

## What needs attention

| #           | Finding                                                                 | Why it matters                                                                                                                                                                                                                                                                                    | Fix                                                                                                                          |
|-------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1<br>High   | <b>The chain holds no money, and one platform key signs everything.</b> | Buyers and farmers still have to trust the platform. The chain is an audit log, not protection.                                                                                                                                                                                                   | Stage 2 escrow.                                                                                                              |
| 2<br>High   | <b>Retries can double-apply.</b>                                        | If a transaction lands but the reply is lost (timeout), the row is marked FAILED and retry sends it again. A repeated attestation appends the same hash twice and changes the chain. A repeated settle fails with 'already settled' and stays FAILED forever. Only commit checks the chain first. | Before resending, read the account (status, evidence count) and skip if already applied. Treat 'already settled' as success. |
| 3<br>High   | <b>Not ready to run on Vercel.</b>                                      | The authority key is read from a file path. Sends happen inside the web request and poll for up to about 20 seconds. The program id is a local-dev key. The payments code is not committed, so the live backend does not include it, and PAYMENT_PROVIDER defaults to mock.                       | Key as an encrypted env secret. Send from a cron or worker. Deploy to devnet with a real program keypair.                    |
| 4<br>Medium | <b>No tests for the program itself.</b>                                 | Only a Python test with a fake RPC exists. It proves the bytes are encoded as expected, not that the program rejects a wrong signer or double settle.                                                                                                                                             | LiteSVM tests: wrong signer, wrong PDA, double settle, bad outcome.                                                          |
| 5<br>Medium | <b>Amount built with possibly-float multiplication.</b>                 | The code computes $\text{int}(\text{quantity\_kg} * \text{price\_per\_kg} * 100)$ . If those fields are floats, a cent can be lost, and this number becomes the on-chain amount. (I have not checked the column types.)                                                                           | Use integer cents or Decimal end to end.                                                                                     |
| 6<br>Medium | <b>The plan and the code disagree.</b>                                  | Plan: seeds [order, id] and 13 instructions. Deployed program: seeds [settlement, hash] and 3 instructions. 'Replace the program' means a new program id and a data migration.                                                                                                                    | Extend the current program with a vault instead of a parallel rewrite.                                                       |
| 7<br>Low    | <b>The plan is big with open questions.</b>                             | Seven phases. Undecided: which token, SPL vs Token-2022, farmer wallets, who pays fees. On-chain offers and merchant profiles add cost for little benefit now.                                                                                                                                    | Answer the four questions on the last page. Skip offers and profiles.                                                        |

Checked: the adapter test passes. The other test in the same file needs Postgres running and errored on this machine only for that reason. The program is built locally but has no Rust tests and was never deployed.

# The Solana-first plan: how it fits

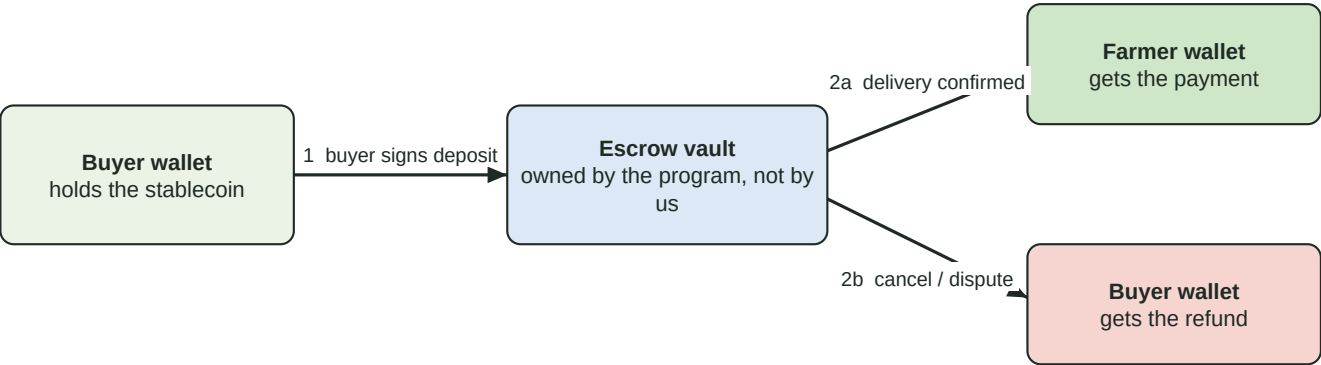
The plan (solana-first/docs) is sound in its main idea: **Solana owns settlement; Postgres owns private data, search and maps**. That matches what the code already does, plus one big addition: real escrow.

## What the plan adds

| Piece                                 | What it does                                                                          | Verdict                                                                 |
|---------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Escrow vault                          | Program-owned account holds buyer funds until release or refund.                      | <b>Do it</b> (stage 2). The main benefit.                               |
| Wallet connect                        | Buyer signs the deposit in their own wallet. Wallet is for signatures, not for login. | <b>Do it</b> (stage 2).                                                 |
| Indexer                               | Copies chain events into Postgres, with a job that checks the two agree.              | <b>Do it</b> (stage 3). Needed once users act on chain without the API. |
| On-chain disputes                     | Resolver decides refund or release on chain.                                          | <b>Later</b> (stage 3). Keep it in the API until real volume.           |
| Offers and merchant profiles on chain | Product listings and profile commitments as accounts.                                 | <b>Skip</b> . Search and listings belong in Postgres.                   |
| 13 instructions, 7 account types      | Full lifecycle on chain.                                                              | <b>Grow gradually</b> . Add an instruction when a stage needs it.       |

## What stage 2 looks like for money

Stage 2: money sits in a vault that only the program controls



The platform key can only press 'release' or 'refund'. It cannot choose another destination, so even a hacked server cannot send escrowed money to itself. That is the real protection stage 2 adds.

Figure 4. Funds move only along the two allowed paths.

# Recommended path

Keep the marketplace working exactly as it is, and add Solana in layers. After each stage the project is shippable.

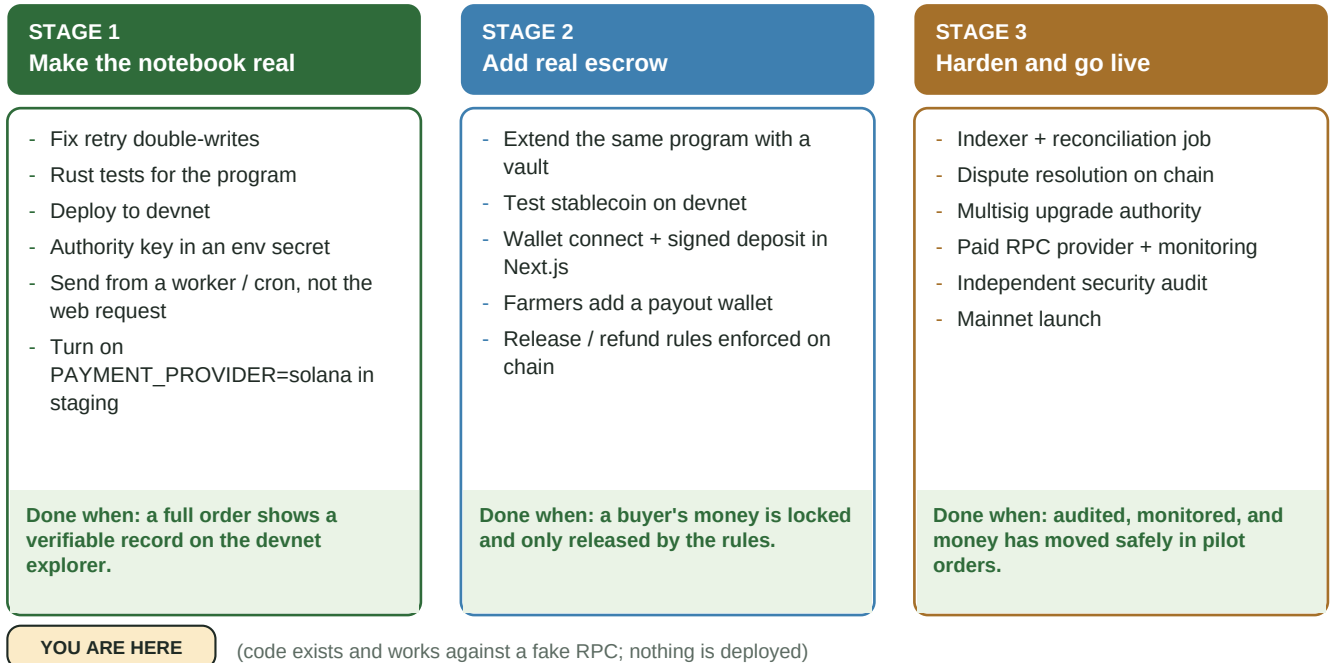


Figure 5. Three stages. Stage 1 is small; stage 2 is the largest and the one that adds real protection.

## Decisions I need from you

| Question                                                     | Why it matters                                                                 | My suggestion                                                                                          |
|--------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 1. Do you want real money on chain, or only an audit trail?  | Decides whether stage 2 happens at all.                                        | Start with the audit trail (stage 1) and decide with devnet results.                                   |
| 2. Which payment token?                                      | Escrow code depends on it. SPL vs Token-2022 must be chosen before writing it. | A regulated stablecoin such as USDC with plain SPL Token. Test with a devnet copy first.               |
| 3. Will farmers have wallets, and who pays network fees?     | A farmer with no wallet cannot be paid on chain.                               | Platform pays fees (fractions of a cent). Farmers add a payout wallet; start with a few pilot farmers. |
| 4. How do users turn local currency into the token and back? | Without an on/off ramp the escrow is unusable for real farmers.                | Pick a provider before stage 2. This is a business question, not a code one.                           |

**Next step if you agree:** I would start stage 1 with the retry fix and the program tests, since they are small and need no decisions from the list above.